



Sesión 3

IA de Agentes y Transformación Organizativa

Alberto Cámara · ISDI MDA · 2026

<> Agenda — Sesión 3

Bloque 1 (80 min)

- Objetivo: automatizar respuestas a reviews
- Del LLM al agente: la diferencia clave
- Memoria, planificación, herramientas
- Multi-agent orchestration
- LangGraph: grafos de estado
- Demo en vivo: agente de atención al cliente

Bloque 2 (80 min)

- De la automatización a la transformación
- Gobernanza de agentes
- ⚠ **Sección crítica:** escepticismo fundado
- Cierre del curso
- Debate + práctica final

Bloque 1

De los LLMs a los Agentes

<> Objetivo: automatizar respuestas a reviews

"Compré unos auriculares hace tres semanas y el sonido del oído derecho ha dejado de funcionar por completo. Quiero una devolución inmediata. Esto es inaceptable."

El agente que vamos a construir hoy:

1. **Clasifica** el ticket: devolución, urgencia alta
2. **Consulta** el catálogo de reviews para recuperar contexto relevante
3. **Redacta** una respuesta empática y específica
4. **Decide** si responder automáticamente o escalar a un humano

→ *Todo esto sin que nadie lo programe caso a caso.*

<> Del LLM al Agente

Un **LLM** recibe texto y devuelve texto.

Un **agente** es un sistema que percibe su entorno y toma acciones para conseguir objetivos.

Un **Agente IA** usa un LLM: pasa la información del entorno en el **contexto** y usa la respuesta del **LLM** para decidir qué acción tomar.

Componente	Qué aporta
Memoria	Contexto de conversaciones pasadas; conocimiento indexado
Planificación	Decide qué pasos dar para resolver una tarea
Herramientas	Acceso a APIs, bases de datos, código ejecutable
Bucle de acción	Actúa, observa el resultado, decide el siguiente paso

<> Memoria: Corto vs. Largo Plazo

Memoria de corto plazo

- El contexto de la conversación actual
- Limitada por la ventana de contexto del modelo
- Se pierde cuando termina la sesión

Memoria de largo plazo

- Información indexada y recuperable
- Persiste entre sesiones

El vector store de reviews de electrónica que indexamos en la Sesión 2 **es** la memoria de largo plazo de nuestro agente de hoy. Las sesiones se encadenan.

<> Planificación: ReAct

ReAct (Reasoning + Acting) es el patrón de planificación más extendido:

- **Observación:** el cliente pide una devolución por fallo en auricular
- **Razonamiento:** necesito información sobre el producto para responder
- **Acción:** consultar el catálogo de reviews (herramienta RAG)
- **Observación:** encontré 4 reviews relevantes sobre auriculares con fallos
- **Razonamiento:** tengo contexto suficiente para redactar una respuesta
- **Acción:** generar respuesta → evaluar si escalar

El modelo razona en voz alta sobre qué hacer antes de hacerlo.

<> Herramientas: Qué Puede Hacer un Agente

Una herramienta es cualquier función que el agente puede invocar:

- **Búsqueda en base de datos** (el RAG)
- **Llamadas a APIs** (sistema de pedidos, CRM, inventario)
- **Ejecución de código** (cálculos, análisis de datos)
- **Navegación web** (verificar información pública)
- **Envío de mensajes** (email, Slack, tickets de soporte)

Las herramientas con **efectos secundarios reales** (enviar email, modificar datos) requieren supervisión adicional. Un agente que puede actuar también puede equivocarse con consecuencias reales.

<> Multi-Agent Orchestration: ¿Por Qué Dividir?

Un solo agente con muchas responsabilidades se vuelve frágil e impredecible.

Ventajas de sistemas multi-agente:

- Cada agente tiene un contexto más pequeño → menos alucinaciones
- Especialización → prompts más precisos
- Paralelización → tareas independientes se ejecutan a la vez
- Fallos aislados → un agente falla sin tumbar el sistema completo

<> Patrones de Orquestación Multi-Agente

Patrón	Descripción	Cuándo usarlo
Pipeline	A → B → C en secuencia	Tareas lineales con pasos bien definidos
Supervisor	Un agente orquestador dirige a N especializados	Cuando la tarea requiere coordinación dinámica
Red de agentes	Cualquier agente puede llamar a otro	Problemas complejos y abiertos

Nuestro agente de hoy usa el patrón **pipeline**: clasificación → RAG → redacción → escalado. Es el punto de entrada natural antes de añadir complejidad.

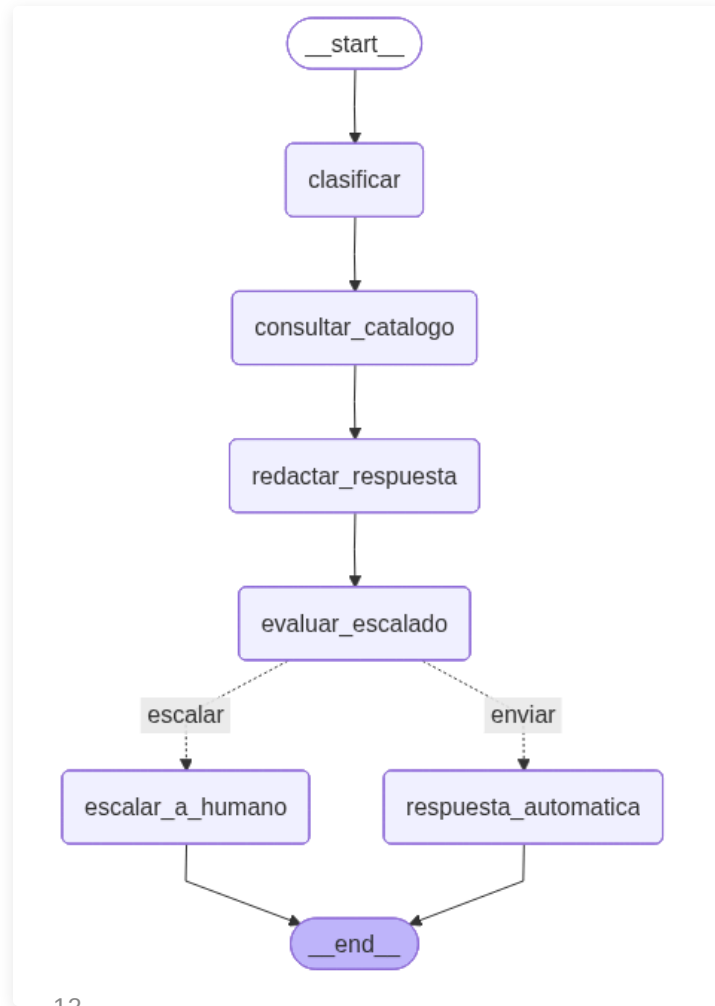
<> LangGraph: Grafos de Estado

LangGraph modela el agente como un grafo dirigido donde:

- **Nodos**: funciones que transforman el estado
- **Aristas**: transiciones entre nodos (fijas o condicionales)
- **Estado**: un diccionario tipado que fluye por el grafo

```
class TicketState(TypedDict):  
    ticket: str          # mensaje original del cliente  
    category: str        # clasificación del ticket  
    urgency: str         # alta | media | baja  
    ...
```

<> El Grafo del Agente de Atención al Cliente



<> Demo en Vivo: El Agente Completo

`demo_agente_atencion_cliente.ipynb`

Lo que vamos a hacer

1. Reutilizar el vector store de la Sesión 2
2. Definir el estado y los nodos del agente
3. Construir y compilar el grafo LangGraph
4. Visualizar el grafo de estados
5. Procesar tickets reales y ver el flujo paso a paso

→ *Cambiar a notebook*



Descanso

15 minutos

Bloque 2

Transformación, Gobernanza y Perspectiva

<> De la Automatización a la Transformación

Automatización (lo que hemos construido):

Un proceso existente ejecutado más rápido o a menor coste.

Transformación (lo que habilita):

Rediseño del proceso completo, nuevas capacidades imposibles antes.

Ejemplo: El agente de atención al cliente no solo responde más rápido.

Permite ofrecer soporte 24/7 en 10 idiomas, personalizado por historial, con análisis de sentimiento en tiempo real — una propuesta de valor que no existía.

<> Nuevos Roles Profesionales

Rol	Qué hace	Por qué es nuevo
AI Engineer	Construye y mantiene pipelines LLM + agentes	Antes no existía la capa de orquestación
Prompt Designer	Diseña y evalúa prompts para producción	La calidad del output depende del input
AI Auditor	Verifica sesgos, alucinaciones y cumplimiento	Regulación + riesgo reputacional
AI Product Manager	Define casos de uso y métricas de éxito	El producto ahora incluye comportamiento del modelo

<> Gobernanza de Agentes: El Problema Real

Un agente en producción puede:

- Tomar decisiones erróneas con consecuencias reales
- Actuar de forma coherente pero inesperada al combinar herramientas
- Escalar su impacto exponencialmente si tiene acceso a APIs

Requisitos mínimos para producción:

- **Logging completo**: cada decisión del agente, con timestamp e inputs
- **Límites de acción**: el agente no puede enviar más de N emails/hora, modificar datos de más de M clientes a la vez
- **Trazabilidad**: dado un output erróneo, reproducir exactamente qué nodos se ejecutaron y con qué estado
- ¹⁸ **Circuit breaker**: mecanismo para detener el agente si detecta anomalías

<> Gobernanza: El Escape Hatch

Regla de diseño: siempre debe existir un camino para que un humano tome el control.

El nodo `escalar_a_humano` de nuestro agente no es un detalle de implementación — es una decisión de diseño fundamental.

En entornos regulados (finanzas, salud, legal) la supervisión humana no es opcional: es un requisito del **EU AI Act** para sistemas de alto riesgo.

Paradoja de la confianza: cuanto más capaz parece el agente, más tentador es quitarle la supervisión. Ese es exactamente el momento de más riesgo.

Sección Crítica

Escepticismo fundado: qué prometen vs. qué demuestran

<> Qué Prometen los LLMs

- Razonamiento general a nivel humano
- Autonomía total en tareas complejas
- Sustitución de profesionales del conocimiento
- AGI inminente

¿Qué demuestran hoy?

- Excelente compresión y síntesis de texto
- Generación fluida y coherente a corto plazo
- Capacidad de seguir instrucciones complejas
- Herramienta de productividad muy potente — con supervisión

⌞ Limitaciones Estructurales que Persisten

Limitación	Descripción
Razonamiento lógico	Fallan en aritmética, lógica formal y problemas de planificación largos
Consistencia	El mismo prompt produce outputs diferentes en distintas ejecuciones
Conocimiento con fecha de corte	No saben lo que no estaba en su corpus de entrenamiento
Calibración de incertidumbre	No distinguen bien lo que saben de lo que no saben
Causalidad	Detectan correlaciones, no causas

Ninguna de estas limitaciones se resuelve con más parámetros o más datos. Son propiedades estructurales de la arquitectura actual.

<> ¿Burbuja...

Señales de burbuja

- Valoraciones desconectadas de ingresos reales
- Promesas de AGI para "los próximos 2 años" (se repite desde 2022)
- Muchos proyectos piloto que no llegan a producción
- Coste energético y económico creciente sin sostenibilidad clara

<> ...o Transformación Real?

Señales de transformación real

- Productividad medida en tareas concretas: +20–50% documentada
- Adopción masiva en código, texto, imagen — cambio de comportamiento real
- Nuevos productos imposibles antes (asistentes médicos, tutores adaptativos)
- Consolidación del sector: los players secundarios desaparecen

<> Una respuesta honesta:

Hay **transformación real en dominios acotados; hype en los más ambiciosos.**

La historia de la tecnología sugiere que ambas cosas son verdad a la vez.

<> Hacia Dónde Va la Tecnología

Tendencias con evidencia (2025–2026)

- **Modelos más pequeños y eficientes**: Qwen3-1.7B, Phi-4-mini — capaces de correr en CPU sin GPU dedicada
- **On-device**: modelos en el teléfono, sin conexión ni coste por token (Apple Intelligence, Gemini Nano)
- **Multimodalidad**: texto + imagen + audio + vídeo como input nativo
- **Agentes con memoria persistente**: el patrón que hemos construido hoy se convierte en estándar

La tendencia real no es "modelos más grandes". Es **modelos más pequeños corriendo más cerca del usuario**, con menos dependencia de infraestructura centralizada.

<> Práctica Final: Diseñad un Sistema Agente

practica_agente_negocio.ipynb

Grupos de 3–4 personas. 20 minutos de diseño + código, 3 min de presentación.

Elegid un proceso de vuestro sector y diseñad un agente LangGraph:

1. ¿Qué estado necesita el agente? (TypedDict)
2. ¿Qué nodos tiene el grafo? (máx. 5)
3. ¿Qué herramientas/memoria necesita cada nodo?
4. ¿Cuándo escala a un humano?
5. ¿Qué métricas de éxito y de gobernanza definiríais?

Bonus: implementad el grafo y corred un caso de prueba.



Cierre del Curso

Las tres sesiones en una frase cada una:

1. Los LLMs son predictores de tokens entrenados a escala — poderosos, frágiles e impredecibles
2. RAG es el puente entre los modelos genéricos y el conocimiento de vuestro negocio
3. Los agentes son LLMs con memoria, planificación y acción — el patrón de la próxima década

Gracias. ¿Preguntas?